File Descriptors, Sockets and a Sequential TCP Server

Lecture 07

Computer Networks and Distributed Programming

Where we are: from reading protocols to writing them

So far every protocol arrived as a description: these messages, in this order, with these fields.

- We read a request and a reply, and said the client sends one and the server answers.
- We never asked what a program actually does to make a message leave the machine.
- That step is a handful of system calls, and they are the same ones for every protocol we have seen.

Today: the calls, and a server that answers a real client.

A network application is a pair of programs

- A network application is normally a client program and a server program, on two different end systems.
- When they run, a client process and a server process exist, and those are what communicate.
- They communicate by reading from and writing to sockets: nothing else crosses the boundary.
- So the developer's job is to write both sides — or one side that conforms to what the other expects.

A program is a file on disk; a process is that file running. The network never sees the first one.

Open protocols and proprietary ones

Open

The rules are published — an RFC or another standards document — so anyone can write either side and the two will interoperate.

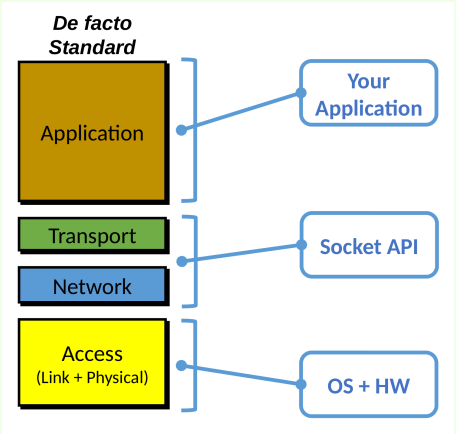
Proprietary

The rules are not published. One team writes both sides and controls the format, and nobody else can write a program that talks to it.

- A browser from one vendor talks to a server from another because both follow the same published rules.
- An open protocol should use its well-known port; your own protocol must stay away from those.

Where the socket sits in the stack

- Above the line, your application: everything there is code you write.
- The socket API is the line itself, and transport and network are already implemented below it.
- Libraries and frameworks wrap sockets and add convenience, but the basic API is still what they call.



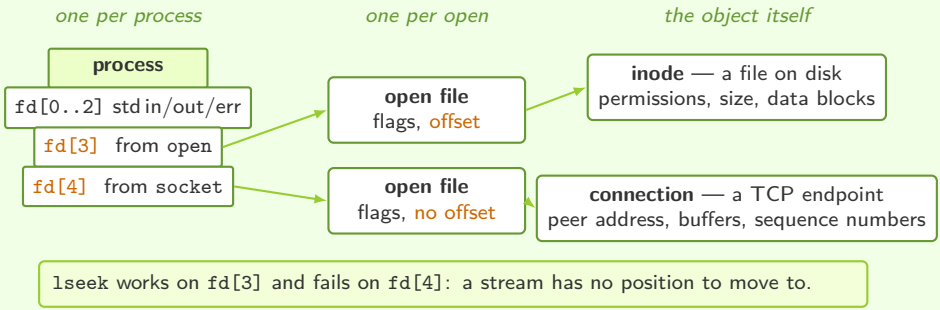
Everything is a file: the descriptor

When a process asks the kernel to open something, it gets back a non-negative integer.

- That integer is the file descriptor: the process's handle on the thing it opened.
- It is what you pass to `read` and `write`; the process never sees anything else.
- Three of them exist before your code runs: 0 standard input, 1 standard output, 2 standard error.
- `STDIN_FILENO`, `STDOUT_FILENO`, `STDERR_FILENO` name them, and are declared in `<unistd.h>`.

What a descriptor actually points to

The integer indexes a table in the process, whose entry points to a shared entry, which points to the object itself.



A socket is one of these. The same integer, obtained from a different call — and that is the whole bridge to what follows.

read and write

Two calls, one descriptor each, and a count of bytes.

```
1 #include <unistd.h>
2
3 ssize_t read (int fd,       void *buf, size_t count);
4 ssize_t write(int fd, const void *buf, size_t count);
```

- `read` returns the number of bytes actually read, 0 at end of input, -1 on error.
- `write` returns the number of bytes actually written, or -1 on error.
- Note: both say actually. Neither promises you `count`.

read can return less than you asked for

You ask for 1000 bytes and get 12. Is that an error?

No. It is the normal case on anything that is not a file on disk: a terminal returns a line, a pipe returns what is in it, a socket returns what has arrived so far.

- The sender's 1000 bytes may still be travelling, split across several packets that arrive at different times.
- A signal can interrupt the call and cut a read short.
- Consequence: on a socket, a single `read` is almost never enough: you loop until you have what you need.
- A return of 0 is different: it is not a short read, it means the other side has closed.

write can write less than you asked for

And if `write` returns 600 out of 1000?

Then 400 bytes have not been sent. The kernel's send buffer was full, and `write` returned what it managed to take.

- Consequence: you must retry with the rest, from the offset the return value gives you.
- On short messages this never happens, which is exactly why it is discovered late and hurts.
- Writing to a socket the peer has already closed is worse: the process gets a signal and dies without a message.
- We will come back to that signal when the code becomes a server that must survive its clients.

When a call fails: −1 and errno

Most system calls report failure the same way, and the reason is left in a global variable.

```c
#include <errno.h>

int fd = open("data.txt", O_RDONLY);
if (fd < 0) {                       /* -1: something went wrong */
    perror("open");                 /* open: No such file or directory */
    exit(EXIT_FAILURE);
}
```

- `errno` holds the code; `perror` prints your string, a colon and the message for that code.
- From here on the slides shorten this to one line — but the check is never the part you drop.

The socket

Definition

A socket is one end of a two-way communication channel between two processes, referred to by a descriptor.

- Introduced in BSD 4.2 in 1983, and still the de facto standard: Unix, macOS, and Windows with a different spelling.
- The two processes may be on the same machine or on two machines across the world: the calls do not change.
- It is not tied to TCP/IP. The same API served other protocol families, and it is why the calls look generic.
- Being a descriptor, it accepts `read`, `write` and `close` — the calls you already know.

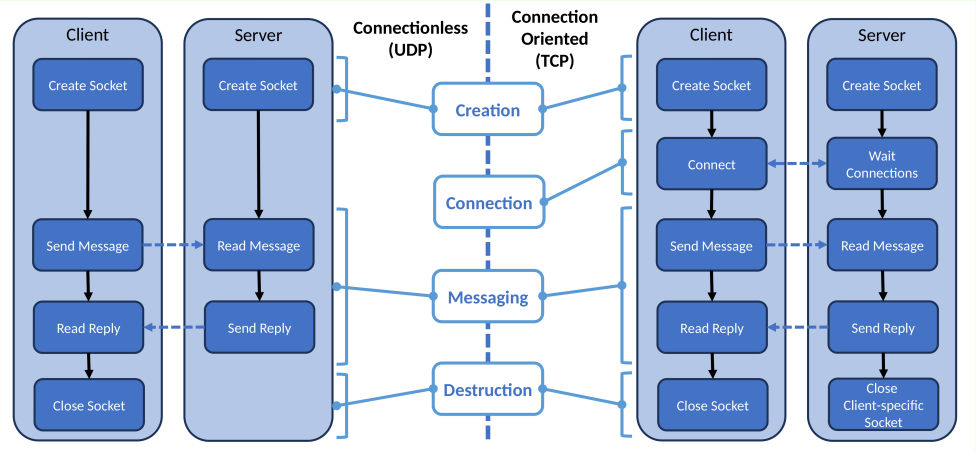
Domain, type, protocol

One generic API; the three arguments of `socket` choose what it means.

Domain — which family	
`AF_INET`	IPv4
`AF_INET6`	IPv6
`AF_LOCAL`	two processes on this machine, through the file system
Type — which kind of channel	
`SOCK_STREAM`	a byte stream: connected, ordered, reliable
`SOCK_DGRAM`	independent messages, no connection, no guarantees
`SOCK_RAW`	the packet built by hand, below the transport layer

The third argument picks a protocol inside the family, and 0 means “the usual one”: TCP for a stream, UDP for datagrams.

Four phases, with and without a connection



Both sides create and destroy; both exchange messages. Only the connection-oriented side has a middle phase, and everything different in the code follows from that one box.

socket(): creating the endpoint

```c
#include <sys/socket.h>

int fd = socket(AF_INET, SOCK_STREAM, 0);   /* IPv4, TCP */
if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
```

- It returns a descriptor, exactly like `open`: the same integer, from a different call.
- The endpoint exists but is anonymous: no address of its own yet, and connected to nothing.
- Everything that follows — bind, connect, listen, accept — takes this descriptor as its first argument.

An endpoint needs an address

```c
struct sockaddr {                   /* the generic one: what the calls take */
    sa_family_t sa_family;
    char        sa_data[14];        /* 14 bytes, meaning set by the family  */
};

struct sockaddr_in {                /* the IPv4 one: what you fill in       */
    short          sin_family;      /* AF_INET                              */
    unsigned short sin_port;        /* port    -.                           */
    struct in_addr sin_addr;        /* address  |- the same 14 bytes        */
    char           sin_zero[8];     /* zeros   -'                           */
};
```

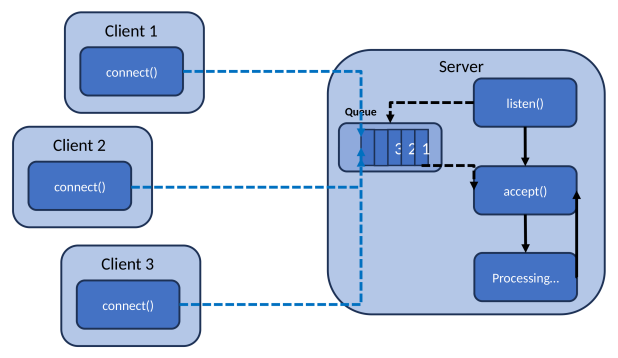
- Every family has its own struct, but the calls take a pointer to the generic one — hence the cast you will see everywhere.
- Filling it in needs `htons` and `inet_pton`, because the network writes numbers its own way. Next time, in full.

bind(): giving the socket a local address

```
1 int bind(int fd, const struct sockaddr *addr, socklen_t len);
```

- It attaches the socket to a local address and port: the place where this process will be reachable.
- Server: required. A client cannot knock on a door whose number nobody agreed on beforehand.
- Client: optional, and normally omitted: the kernel assigns a free port when the connection is opened.
- Returns 0 or -1. The failure you will meet is address already in use, when the port is still taken.

listen(): the queue of pending connections



- `listen` does not wait for anything: it turns the socket passive and opens a queue behind it.
- Connections complete and queue up on their own; `accept` takes the first one out, one call per client.

listen() and accept()

```
1 int listen(int fd, int backlog);
2 int accept(int fd, struct sockaddr *addr, socklen_t *len);
```

- `backlog` is how many completed connections may wait before the kernel starts refusing or ignoring them.
- `accept` blocks until there is one, then returns a brand-new descriptor — not a return code.
- Through `addr` it also tells you who connected; pass `NULL` if you do not care.
- `len` goes in as the size of your struct and comes back as the size actually used: it is a pointer for that reason.

The server holds two kinds of socket

- The listening socket is created once, bound to the known port, and never carries data.
- Every `accept` returns a connection socket, dedicated to one client and useless to any other.
- Data flows only on the second kind. The first one keeps listening while the conversation happens elsewhere.
- So a server serving three clients holds four descriptors, and closing the wrong one is the classic mistake.

The three-way handshake that creates the connection happens in the transport layer, and is invisible to both programs.

connect(): the client side

```
1 int connect(int fd, const struct sockaddr *addr, socklen_t len);
```

- The client needs no `bind`, no `listen` and no `accept`: one call does all of it.
- `connect` returns only when the connection is established — or when it has failed.
- Between those two moments the handshake runs, and the process simply waits: your code has no view of it.
- After it returns, the same descriptor is a two-way channel to the server's connection socket.

Reading and writing on a connected socket

```
1 n = write(fd, "hello\n", 6);        /* no address: it is already fixed */
2 n = read (fd, buf, sizeof buf);
```

- Once connected, the descriptor behaves like any other: the calls from the beginning of this lecture work unchanged.
- No destination is passed, because the pair of addresses was fixed when the connection was established.
- `send` and `recv` are the socket-specific versions: same arguments plus a flags field, which we do not need yet.
- Both still return how much was actually transferred — the two cases we saw are now on the critical path.

close(), and how the peer learns you are gone

- `close` says that this process is done with the descriptor, and releases it.
- On a connection it also starts the shutdown, and the peer's next `read` returns 0.
- That 0 is the only end-of-message the API gives you for free, and it costs you the connection.
- Closing is symmetric: the connection is really over when both sides have closed it.

Whose descriptor is being closed?

A server that closes the listening socket instead of the connection socket serves one client and then accepts nobody.

The client, end to end

```c
int main(void) {
    char buf[256];
    struct sockaddr_in srv;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    memset(&srv, 0, sizeof srv);
    srv.sin_family = AF_INET;
    srv.sin_port   = htons(5000);
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);   /* not INADDR_ANY */
    if (connect(fd, (struct sockaddr *) &srv, sizeof srv) < 0) {
        perror("connect"); exit(EXIT_FAILURE); }
    write(fd, "hello\n", 6);
    ssize_t n = read(fd, buf, sizeof buf - 1);  /* room for the '\0' */
    if (n > 0) { buf[n] = '\0'; printf("%s", buf); }
    close(fd);
}
```

Four calls do the work: `socket`, `connect`, `write`, `read`. The rest is filling in an address.

The server: setting up the listening socket

```c
struct sockaddr_in me;
int yes = 1;
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
memset(&me, 0, sizeof me);
me.sin_family      = AF_INET;
me.sin_port        = htons(5000);
me.sin_addr.s_addr = htonl(INADDR_ANY);   /* any local interface */
if (bind(listen_fd, (struct sockaddr *) &me, sizeof me) < 0) {
    perror("bind"); exit(EXIT_FAILURE); }
if (listen(listen_fd, 16) < 0) {
    perror("listen"); exit(EXIT_FAILURE); }
```

`SO_REUSEADDR` is what lets you restart the server immediately instead of waiting for the old port to be released.

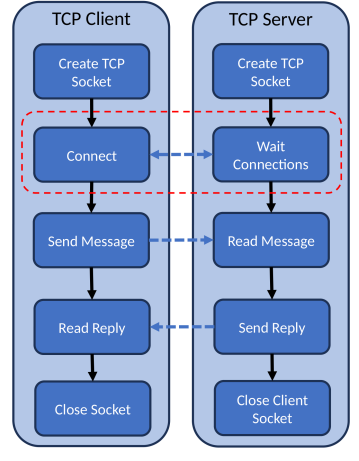
The server: the accept loop

```c
static void serve(int fd) {              /* one whole conversation */
    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        if (write(fd, buf, n) != n) break;      /* short write: give up */
    if (n < 0) perror("read");
}

for (;;) {
    struct sockaddr_in cli;
    socklen_t len = sizeof cli;          /* socklen_t, not int */
    int fd = accept(listen_fd, (struct sockaddr *) &cli, &len);
    if (fd < 0) { perror("accept"); continue; }
    serve(fd);
    close(fd);                           /* the client's socket, not ours */
}
```

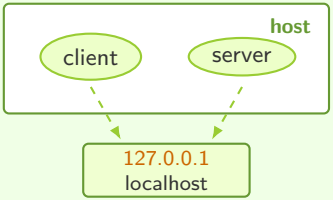
The two flows side by side

- Both columns begin the same way and end the same way: create, exchange, close.
- The dashed box is the only asymmetric step — one side calls, the other waits.
- On the right, close client socket: the server closes the connection, not the door.



Running both on one machine

```
$ ./server &
$ ss -tln | grep 5000
LISTEN 0 16 0.0.0.0:5000 0.0.0.0:*
$ nc 127.0.0.1 5000     not your code
hello
hello
$ ss -tn | grep 5000
ESTAB 0 0 127.0.0.1:47122 127.0.0.1:5000
```



nothing reaches the wire

Test the server with `nc` before writing a client: if it fails, you know which of the two programs is wrong.

A socket carries bytes, not types

You wrote `write(fd, &x, sizeof x)` and it compiled. It will still be wrong on the other machine.

- Size: `int` is not the same width everywhere: fixed-width types (`uint32_t`) are the only safe ones on the wire.
- Order: the two machines may store the bytes of a number in opposite orders — next lecture's subject.
- Boundaries: nothing in the stream says where one value or one message ends. That one is yours to solve.

The simplest answer to all three is to send text, and it is what HTTP, SMTP and the DNS query format chose.

Encoding a message as text

```
char msg[128] = "";                  /* sending: numbers -> one line */
for (int i = 0; i < n; i++) {
    char field[16];
    snprintf(field, sizeof field, "%d ", a[i]);
    strncat(msg, field, sizeof msg - strlen(msg) - 1);
}
write(fd, msg, strlen(msg));

char *tok = strtok(line, " ");       /* receiving: line -> numbers */
while (tok != NULL) {
    vals[k++] = atoi(tok);
    tok = strtok(NULL, " ");
}
```

Both sides must agree on the separator and on the order of the fields. That agreement is the application-layer protocol.

Where does a message end?

The client sends two lines. How many `read` calls does the server need?

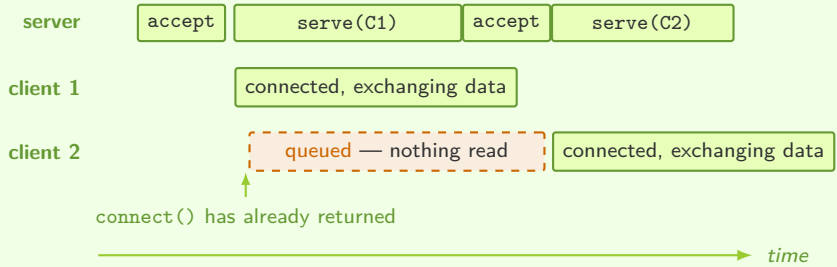
Unknown. It may be one call returning both lines, or three calls returning pieces of the first. TCP is a byte stream and preserves order, nothing else.

- Your `write` calls are not messages: the boundaries between them are not transmitted.
- So the receiver cannot know a message is complete unless the protocol tells it.
- Two ways, and every protocol picks one: a delimiter to scan for, or a length announced first.
- HTTP does both — a blank line ends the header, then `Content-Length` counts the body.

One client at a time

Two clients connect a second apart. What does the second one see?

Its `connect` succeeds immediately — the kernel completed it — and then nothing, until the server returns from serving the first.



The server is not slow: it is busy inside `serve`, and one client that never sends anything blocks every other one.

Summary

- A descriptor is the process's handle on something the kernel opened, and a socket is one of them.
- `read` and `write` return how much was actually transferred: on a socket, that is rarely what you asked for.
- `socket`, `bind`, `listen`, `accept` on the server; `socket` and `connect` on the client.
- The server keeps a listening socket that never carries data and one connection socket per client.
- A stream carries bytes: message boundaries do not exist unless the protocol creates them.
- A server that serves one client at a time is correct and unusable, and that is where we go next.

The project: one service, nine versions

For the rest of the course you will write one program, not a new exercise every week: a line-oriented chat service.

- Every version keeps the previous one working and removes one of its limits.
- Concurrency comes first — processes, threads, one process watching many connections at once, a pool of workers.
- Then the protocol: real message framing and error codes, and a version over datagrams that must survive loss.
- Then you look at it from outside: the packets it generates, and what it costs under load.

Each limit is discovered by running the previous version, not by being told about it.

Iteration 1: a sequential TCP chat

What to write

A server that accepts one client at a time and, for every line it receives, sends back that line prefixed with a counter; a client that reads lines from the terminal and prints what comes back.

- Lines are separated by `\n`, and a line is complete only when you have seen one: loop on read.
- The client stops at end of input and closes; the server notices the 0 and goes back to `accept`.
- Names: the server is `chatd`, the client is `chat`: the same two, unchanged, for all nine versions.
- Test: run it on `127.0.0.1`, check the server with `nc` first, then with your own client.

Then open a second client and answer: what does it see, and what does `ss -tn` show while it waits?